

The installation of Fedora 22 alongside Windows in UEFI is slightly tricky for beginners because if you try to boot it in live mode from a bootable pen drive, it just won't work. So, to create a live disk of Fedora 22, the use of LiveUSB Creator is recommended. LiveUSB Creator is a tool specially used to create live disks which support UEFI.

Steps to be followed during the dual boot of Fedora 22 and Windows

1. Turn off *Secure boot* from the boot options in the BIOS settings.
2. Insert the live disk and then proceed with the installation process.
3. In the partitioning stage, make sure that you select manual partitioning instead of automatic partitioning.
4. In manual partitioning, there will be an option to automatically allocate the free space to the operating system or you can also do it manually.
5. If you are doing it manually, make sure to allocate a partition of around 500MB for the */boot/efi* partition where Grub will be installed.
6. Then create a new user and proceed to install and reboot the system.

You have now successfully installed Fedora 22 in your notebook/laptop with UEFI enabled.

Yum to DNF in Fedora 22

After the installation of Fedora 22, when you try to do an update of all the packages from the terminal, you will see something like what's shown in Figure 2, on the terminal screen.

Previously, the Yum (Yellowdog Updater Modified) package installer for Fedora managed the packages in RPM format, so it became the package manager of choice for several RPM-based distributions. But Yum is fairly slow in performance and has high memory usage; code *cruff* has increased the overall time as well. So, from the release of Fedora 22 onwards the Fedora community has decided to use DNF (Dandified Yum) as the default package manager.

The reason for initiating the DNF project was because of the three big drawbacks of Yum: undocumented API, broken dependency solving algorithm, and the inability to refactor internal functions. The last issue is connected with the lack of documentation.

DNF (Dandified Yum)

DNF is a forked version of Yum that uses the *libsolv* library via Hawkey. There are a few more packages included in DNF, which increase its performance. These packages are based on Python. DNF offers some backward compatibilities with Yum.

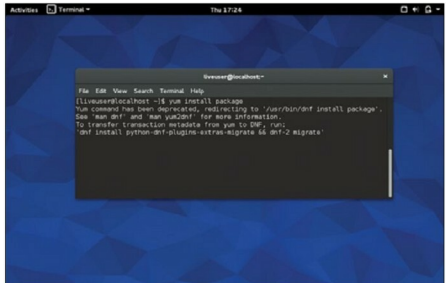


Figure 2: Using yum to install software



Figure 3: Using DNF install software

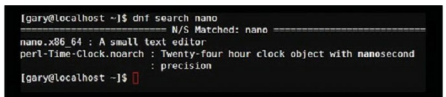


Figure 4: Using DNF to search package

Desktop users using graphical package management tools like the software application won't feel a difference.

If you're using the Yum command in a terminal, you will be reminded that Yum is deprecated and that you should use the DNF command instead. DNF mostly works about the same as Yum. Most of the commands in DNF are similar to Yum commands.

The Hawkey *depsolving* library on which DNF is built is well designed, optimised and is itself built on a successful, low-level SAT library that SUSE has been using for years at production level workloads. The downloading and metadata parsing component used by DNF, *librepo*, is also well designed and complements the Hawkey API nicely.

Rather than use the DNF framework directly, *packagekit* (PK) uses *librepo* and Hawkey to share 80 per cent of the mechanism between PK and DNF. DNF is well organised with unit tests and lots of older compatibility *cruff* removed,